

Variables en C#

Fundamentos de almacenamiento de datos, gestión de memoria e integración práctica en software moderno.

Acerca de este Capítulo

Este material didáctico ha sido diseñado como una guía técnica completa y autosuficiente. Partiendo desde los fundamentos absolutos de la arquitectura de la memoria de la computadora hasta la implementación de patrones de desarrollo profesional en sistemas industriales, financieros y backend enterprise.

CURSO

Curso de Programación C# NET

ENFOQUE TÉCNICO

Desarrollo de C# y Arquitectura

NIVEL EDUCATIVO

Desde Principiante hasta Práctico

REQUISITOS PREVIOS

Ninguno (Lógica desde Cero)

1. Objetivos del Capítulo

El objetivo fundamental de este capítulo es proporcionar al estudiante el conocimiento teórico y la destreza práctica para manipular información en la memoria utilizando C#. Al finalizar la lectura y ejercitación, el estudiante comprenderá exactamente cómo interactúa el software con la memoria RAM del sistema.

Al finalizar este capítulo serás capaz de:

- **Comprender y explicar** qué es una variable y cómo se reserva espacio en la memoria RAM del equipo.
- **Seleccionar el tipo de dato óptimo** (int, double, string, bool, decimal) según el requerimiento técnico del negocio.
- **Declarar, inicializar y reasignar variables** cumpliendo con las reglas sintácticas estricta de C#.
- **Aplicar las convenciones profesionales de nomenclatura** (CamelCase y PascalCase) para escribir código limpio y mantenible.
- **Identificar, diagnosticar y corregir** errores comunes de compilación y ejecución como desbordamientos (overflow) y variables no asignadas.
- **Construir programas reales** para monitoreo de sensores IoT, procesamiento contable e inventarios informáticos.

2. Introducción al Tema

¿Qué es una Variable en C#?

En el mundo de la programación, una **variable** es un espacio delimitado en la memoria RAM del ordenador al cual se le asigna un nombre simbólico (identificador) y un tipo de dato. Sirve como un contenedor reutilizable capaz de almacenar valores durante la ejecución de un programa.

Analogía de la Vida Real

Imagina un gran almacén de logística equipado con miles de *cajas de cartón de distintos tamaños*:

1. La Etiqueta

Es el **nombre** de la variable. Permite identificar la caja rápidamente sin mirar adentro.

2. El Tamaño / Tipo

Es el **tipo de dato**. Una caja para líquidos no sirve para guardar zapatos. Especifica qué cabe.

3. El Contenido

Es el **valor**. Lo que colocas adentro. Puede reemplazarse por otro contenido del mismo tipo.

¿Por qué existe y qué problema resuelve?

Los procesadores (CPU) sólo procesan datos almacenados en direcciones binarias de memoria RAM (ejemplo: `0x7FFF5FBFFD98`). Sería prácticamente imposible para los seres humanos escribir software complejo recordando direcciones numéricas hexadecimales. Las variables resuelven esto permitiendo trabajar con nombres legibles como `temperaturaSensor` o `saldoCuenta`.

¿Qué ocurriría si este concepto no existiera?

Sin variables, los programas serían **estáticos e inmutables**. Un software no podría reaccionar a la entrada de un usuario, no podría almacenar la velocidad de un dron en tiempo real, ni calcular la factura de una compra. La informática moderna se basa enteramente en procesar datos cambiantes.

¿Dónde se utiliza en proyectos reales?

Las variables se utilizan en el 100% de los desarrollos informáticos:

Sector Aplicado	Ejemplo de Uso de Variables	Tipo de Dato Típico
Automatización Industrial / PLC	Almacenar la temperatura de un horno de fundición.	float o double
Sistemas de Finanzas / Comercio	Guardar el precio con precisión exacta de centavos.	decimal
Robótica / Drones	Estado de los motores (Encendido / Apagado).	bool
Gestión de Inventarios	Cantidad física de productos en stock.	int

3. Arquitectura Interna: ¿Cómo funciona en Memoria?

C# es un lenguaje fuertemente tipado (Strongly Typed). Esto significa que la computadora debe saber con exactitud antes de ejecutar el programa cuánta memoria requiere cada variable.



Concepto Clave: Fuertemente Tipado

Una vez que declaras una variable en C# con un tipo de dato específico (por ejemplo, `int` para enteros), no puedes guardar en ella texto o decimales. Esto previene errores graves en tiempo de ejecución.

4. Tipos de Datos Primitivos Principales

Para trabajar con variables, primero debemos conocer las distintas "cajas" que nos proporciona C# para almacenar información básica:

Tipo	Espacio	Rango / Formato	Uso Recomendado
int	4 Bytes (32 bits)	-2,147,483,648 a 2,147,483,647	Contadores, cantidades enteras, IDs.
long	8 Bytes (64 bits)	-9×10 ¹⁸ a 9×10 ¹⁸	Grandes números, poblaciones, ticks.
float	4 Bytes (32 bits)	7 dígitos de precisión	Gráficos 3D, sensores no analíticos.
double	8 Bytes (64 bits)	15-17 dígitos de precisión	Cálculos científicos, física, distancias.
decimal	16 Bytes (128 bits)	28-29 dígitos de precisión alta	Finanzas, moneda, transacciones bancarias.
bool	1 Byte (8 bits)	true o false	Banderas de control, interruptores, estados.
char	2 Bytes (16 bits)	Un caracter Unicode (ej. 'A')	Códigos de teclado, símbolos únicos.
string	Variable	Texto (secuencia de caracteres)	Nombres, mensajes, correos, descripciones.

5. Estructura y Sintaxis de una Variable

Para usar una variable en C#, se sigue una regla sintáctica fundamental compuestas por tres o cuatro elementos:

```
// Sintaxis General:  
tipoDeDato nombreVariable = valorInicial;
```

1. Declaración

Reservar el espacio en memoria informando el tipo y nombre.

```
int velocidad;
```

2. Asignación / Inicialización

Guardar el primer valor dentro del espacio reservado.

```
velocidad = 80;
```

Ambos pasos se pueden combinar en una sola línea de código limpia:

```
int velocidad = 80; // Declaración e inicialización simultánea
```

6. Ejemplos Progresivos de Código

Nivel 1 — Ejemplo Básico: Presentación Personal

Objetivo: Declarar e imprimir los tipos de datos primarios en la consola.

```
using System;

class Program
{
    static void Main()
    {
        string nombre = "Carlos";
        int edad = 28;
        double altura = 1.75;
        bool esEstudiante = true;

        Console.WriteLine("Nombre: " + nombre);
        Console.WriteLine("Edad: " + edad);
        Console.WriteLine("Altura: " + altura + "m");
        Console.WriteLine("¿Es estudiante?: " + esEstudiante);
    }
}
```

Explicación Línea por Línea (Nivel 1):

- `string nombre = "Carlos";` : Reserva memoria para texto y guarda la cadena "Carlos". Se usan comillas dobles `""`.
- `int edad = 28;` : Reserva 4 bytes para guardar un número entero positivo (28).
- `double altura = 1.75;` : Reserva espacio decimal con precisión flotante. Se usa punto `.` como separador decimal.
- `bool esEstudiante = true;` : Guarda un valor lógico (Verdadero).
- `Console.WriteLine(...);` : Imprime en la consola el texto concatenado con el valor actual de cada variable.

Nivel 2 — Ejemplo Intermedio: Calculadora de Nómina Básica

Objetivo: Reasignar variables y realizar operaciones matemáticas simples entre tipos numéricos.

```
double horasTrabajadas = 40.0;
double pagoPorHora = 15.50;

// Cálculo aritmético y almacenamiento en una nueva variable
double sueldoBruto = horasTrabajadas * pagoPorHora;

// Reasignación de valor (Aumento de horas extra)
horasTrabajadas = 45.0;
double sueldoAjustado = horasTrabajadas * pagoPorHora;

Console.WriteLine("Sueldo inicial: $" + sueldoBruto);
Console.WriteLine("Sueldo ajustado: $" + sueldoAjustado);
```

Nivel 3 — Ejemplo Práctico: Control de Sensor de Temperatura Industrial

Objetivo: Aplicar variables en una simulación de automatización de procesos mediante lógica condicional básica.

```
string sensorId = "TEMP-PLC-004";
double temperaturaActual = 88.5; // En grados Celsius
double limiteCritico = 85.0;
bool alarmaActivada = temperaturaActual > limiteCritico;

Console.WriteLine("=== MONITOREO DE PLANTA EN TIEMPO REAL ===");
Console.WriteLine("ID Sensor: " + sensorId);
Console.WriteLine("Temperatura Lectura: " + temperaturaActual + " °C");
Console.WriteLine("Estado de Alarma: " + alarmaActivada);
```

Nivel 4 — Ejemplo Profesional: Telemetría de Dron Agrícola

Contexto: Módulo de un software empaquetado para el monitoreo en tiempo real de vehículos aéreos no tripulados.

```
using System;

namespace TelemetriaDron
{
    class ModuloTelemetria
    {
        static void Main()
        {
            // Identificación del dispositivo
            string codigoDron = "AGRI-DRONE-X1";
            int porcentajeBateria = 82;

            // Telemetría de vuelo
            double altitudMetros = 45.8;
            float velocidadKmh = 24.3f; // Requiere sufijo 'f'

            // Precisión GPS de grado profesional
            double latitud = 19.432608;
            double longitud = -99.133209;

            // Estado de seguridad
            bool retorneAHome = porcentajeBateria < 20;

            // Salida estructurada del sistema
            Console.WriteLine("=== TELEMETRÍA DRON: " + codigoDron + " ===");
            Console.WriteLine("Ubicación: " + latitud + ", " + longitud);
            Console.WriteLine("Altitud/Velocidad: " + altitudMetros + "m | " + velocidadKmh + "kmh");
            Console.WriteLine("Batería Restante: " + porcentajeBateria + "%");
            Console.WriteLine("¿Activar retorno de emergencia?: " + retorneAHome);
        }
    }
}
```

7. Comparaciones Importantes

Es común que los estudiantes principiantes confundan tipos de datos que parecen cumplir la misma función. A continuación se detallan las diferencias estructurales:

Concepto A	Concepto B	Diferencia Principal	Cuándo Usar Cada Uno
double	decimal	double usa coma flotante binaria (rápida, imprecisa). decimal usa base 10 (exacta).	Usa double para física/juegos; usa decimal para precios y contabilidad.
float	double	float ocupa 4 bytes (7 dígitos). double ocupa 8 bytes (15 dígitos).	Usa float si la memoria es limitada (ej. gráficos móviles); double por defecto.
string	char	string almacena texto entre "". char almacena UN SOLO carácter entre ''.	Usa string para frases/nombres; char para opciones ('S'/'N').

8. Errores Comunes al Trabajar con Variables

Error 1: Uso de variables no asignadas (CS0165)

Código Incorrecto:

```
int total;
Console.WriteLine(total); // ¡ERROR DE COMPILACIÓN!
```

Causa y Solución: C# no permite leer una variable que no contenga un valor inicial. Debes asignarle un valor antes de leerla (`int total = 0;`).

Error 2: Desbordamiento de tipo (Overflow)

Código Incorrecto:

```
byte nivel = 250;
nivel = nivel + 10; // Un 'byte' solo soporta hasta 255
```

Causa y Solución: Intentar guardar un número mayor al rango permitido por el tipo de dato ocasionará una pérdida de datos o una excepción. Usa un tipo con mayor rango como `int` .

Error 3: Confusión de comillas (Comillas simples vs Dobles)

Usar `string nombre = 'Juan';` es un error de sintaxis. Los textos (`string`) exigen comillas dobles `""` , mientras que los caracteres (`char`) usan comillas simples `''` .

9. Buenas Prácticas de Programación Profesional

1. Regla CamelCase

Nombra las variables locales iniciando con minúscula y capitalizando cada palabra subsiguiente.

Correcto: `sueldoEmpleado`

Incorrecto: `SueldoEmpleado`

2. Nombres Descriptivos

Evita nombres abstractos de una sola letra salvo en contadores iterativos muy breves.

Correcto: `temperaturaSensor`

Incorrecto: `t o x1`

Sufijos OBLIGATORIOS para Decimales y Flotantes

C# interpreta por defecto cualquier número con decimales como `double`. Para especificar que deseas usar `float` o `decimal`, debes agregar su letra sufijo:

- `float precioBase = 19.99f;` (Sufijo 'f' o 'F')
- `decimal totalFactura = 1500.50m;` (Sufijo 'm' o 'M' por Money)

10. Conectando Conocimientos

Hasta este punto has aprendido cómo C# almacena valores individuales en la memoria. Este concepto es el pilar de construcción para los siguientes bloques del curso:

- **Estructuras Condicionales (Capítulo 2):** Tomarás decisiones en el programa evaluando el valor guardado en una variable `bool` o comparando números (ej. `if (temperatura > 80)`).
- **Bucles e Iteraciones (Capítulo 3):** Utilizarás variables como contadores y acumuladores para repetir procesos automáticamente.
- **Métodos y Funciones (Capítulo 4):** Pasarás variables como argumentos a bloques de código reutilizables.

11. Ejercicios Prácticos (Sin Solución Inmediata)

Pon a prueba tus conocimientos antes de avanzar al mini proyecto. Resuelve los ejercicios en tu entorno de desarrollo (Visual Studio o VS Code).

Ejercicio 1 — Ficha de Producto de Tienda

Objetivo: Practicar selección de tipos de datos adecuados.

Enunciado: Declara e inicializa variables para los siguientes datos de un teléfono celular: Marca, Modelo, Unidades en Almacén, Precio de Venta (con precisión exacta), y si tiene Garantía Extendida. Imprímelos por consola.

Pista: Recuerda usar el sufijo 'm' para la moneda.

Ejercicio 2 — Conversor de Unidades Térmicas

Objetivo: Reasignación y fórmulas matemáticas con variables.

Enunciado: Crea una variable con la temperatura actual de un proceso químico en grados Celsius (`double`). Escribe la fórmula para convertir esa temperatura a Fahrenheit y guarda el resultado en una segunda variable. (Fórmula: $(\text{Celsius} * 9/5) + 32$).

Ejercicio 3 — Detección y Corrección de Errores

Objetivo: Identificar fallos de compilación.

Encuentra los 3 errores sintácticos en el siguiente código:

```
int puntaje = "100";
char inicial = "A";
decimal costo = 25.99;
```

Ejercicio 4 — Reto Complejo: Sistema de Peaje Automatizado

Enunciado: Un peaje cobra según el tipo de vehículo. Diseña un script declarando: Placa (texto), Ejes (entero), Peso en Toneladas (decimal flotante), y TagElectrónico (booleano). Calcula la tarifa básica multiplicando el número de ejes por \$45.50 y muestra el ticket en la pantalla.

12. Mini Proyecto del Capítulo

Proyecto: Sistema de Control de Vuelo de un Dron de Entregas

Contexto: Te han contratado para desarrollar el prototipo inicial del software a bordo de un dron de mensajería urbana.

Requisitos Funcionales:

- Registrar la identificación del paquete y las coordenadas de entrega.
- Monitorear la carga útil (peso del paquete) y la autonomía restante.
- Calcular si el dron está sobrecargado según un límite máximo prefijado.

Guía de Desarrollo Paso a Paso:

Paso 1: Configuración de Variables de Identificación

Crea las variables para el código del paquete (`string`), el peso en kilogramos (`double`) y la distancia al destino en kilómetros (`double`).

Paso 2: Parámetros del Vehículo (Constantes y Límites)

Declara el límite máximo de peso soportado por el dron (ejemplo: `3.5` kg) y el nivel actual de la batería del dron en porcentaje (`int`).

Paso 3: Evaluación del Estado del Vuelo

Declara una variable booleana llamada `esVueloSeguro` que almacene el resultado de verificar si el peso del paquete es menor o igual al límite permitido AND la batería es mayor al 20%.

Paso 4: Generación de la Ficha de Despegue

Muestra en consola un informe claro simulando la pantalla del operador con todos los datos calculados.

Desafío Opcional para el Estudiante

Modifica los valores de peso iniciales para probar ambos casos: un vuelo autorizado y un vuelo rechazado por sobrepeso.

13. Comprueba tus Conocimientos (Autoevaluación)

Responde las preguntas en un cuaderno o documento separado antes de consultar las respuestas explicadas en la siguiente página.

1. ¿Cuál es la diferencia técnica fundamental entre declarar una variable e inicializarla?
2. Si necesitas almacenar la cantidad de habitantes de una ciudad grande (ej. 8,000,000), ¿qué tipo de dato numérico entero debes usar y por qué no deberías usar un `short` ?
3. ¿Por qué el tipo de dato `decimal` es superior a `double` para guardar dinero, a pesar de usar más memoria?
4. ¿Qué ocurre si declaras la variable `int edad = 20;` e intentas ejecutar la línea `edad = "Veintiuno";` más abajo? Explica el concepto involucrado.
5. ¿Cuál es el valor predeterminado impreso en pantalla al intentar leer una variable local no asignada?
6. ¿Qué sufijo sintáctico requiere una variable de tipo `float` al ser asignada con un número decimal literal?
7. Indica qué convención de nombres (CamelCase o PascalCase) se recomienda en C# para declarar variables locales.
8. Analiza el siguiente código: `char simbolo = 'AB';` . ¿Es válido? Justifica tu respuesta.
9. ¿Cuántos bytes de memoria ocupa una variable de tipo `int` en la arquitectura estándar de C#?
10. ¿Qué palabra clave/tipo de dato usarías para registrar si una válvula industrial está abierta o cerrada?

14. Respuestas y Explicaciones de la Autoevaluación

1. Declaración vs Inicialización: La declaración reserva el espacio de memoria indicando tipo y nombre (`int x;`). La inicialización es el acto de asignarle su primer valor (`x = 10;`).

2. Habitantes de Ciudad: Se debe usar `int` (alcanza hasta ~2.1 mil millones). El tipo `short` solo soporta hasta 32,767, por lo que causaría un error de desbordamiento (overflow).

3. Precision de Decimal vs Double: `decimal` usa representación de base 10 sin redondeos binarios inexactos, garantizando precisión exacta de centavos indispensable en contabilidad.

4. Fuertemente Tipado: El compilador arroja un error de tipo (CS0029) porque C# es fuertemente tipado y no permite asignar cadenas de texto a variables declaradas como enteros.

5. Variables no Asignadas: Ninguno. El programa **no compilará**. C# impide expresamente la lectura de variables locales sin inicializar.

6. Sufijo Float: El sufijo `f` o `F` (ejemplo: `float x = 1.5f;`).

7. Convención para Variables: Se utiliza **camelCase** (ejemplo: `totalDescuento`).

8. Análisis de Char: No es válido. El tipo `char` solo puede contener **un único carácter** y debe usar comillas simples. 'AB' contiene dos caracteres.

9. Memoria de Int: Ocupa exactamente **4 Bytes (32 bits)**.

10. Válvula Industrial: El tipo `bool` (booleano), asignando `true` para abierta y `false` para cerrada.

15. Resumen del Capítulo

Concepto	Para qué sirve	Cuándo utilizarlo	Error común a evitar
<code>int</code>	Almacenar números enteros.	Conteo de elementos, índices, cantidades.	Tratar de guardar números con decimales.
<code>double</code> / <code>decimal</code>	Almacenar números racionales/decimales.	Física/Gráficos (<code>double</code>) o Dinero/Precios (<code>decimal</code>).	Olvidar el sufijo 'm' al declarar un decimal.
<code>string</code>	Almacenar texto libre.	Nombres, direcciones, mensajes de interfaz.	Encerrar el texto entre comillas simples ' '.
<code>bool</code>	Almacenar estados binarios.	Interruptores, validaciones, banderas de control.	Asignar cadenas como "true" en lugar de palabras clave.

Ideas Clave que Debes Recordar:

- Una variable es una celda con nombre en la memoria RAM del equipo.
- C# exige definir el tipo de dato antes de usar cualquier variable.
- El tipo `decimal` es indispensable para operaciones de dinero.
- Las variables locales deben inicializarse antes de ser leídas.
- Sigue siempre la convención `camelCase` para un código limpio profesional.

16. Mapa de Ruta de Aprendizaje



¡Felicidades por completar el Capítulo 1! Estás listo para avanzar al **Capítulo 2: Operadores y Toma de Decisiones**.